

intermediate results. Like persisting, checkpointing helps avoid computation, thus minimizing the cost of failure, and avoids recomputation by breaking the lineage graph.

PERSIST AND CACHE

Persisting an RDD means materializing an RDD (usually by storing it in-memory on the executors), for reuse *during the current job*. Spark remembers a persisted RDD's lineage so that it can recompute it for the duration of a Spark job if one of the persisted partitions is lost. After the job ends, the `persist` function takes a `StorageLevel` argument that specifies how the RDD should be stored. Spark provides a number of different storage levels as constants, but each one is created based on five attributes of how to store the RDD: `useDisk`, `useMemory`, `useOfHeap`, `deserialized`, and `replication`. Calling `toString` on a storage level will reveal what options it contains. The [Spark documentation about persistence](#) includes a fairly comprehensive list of the out-of-the-box storage options that are exposed to you.

Still, we think it may useful to provide some more information about each of the five properties that compose each storage option. This should give you a deeper understanding of which option to choose:

`useDisk`

If set, partitions that do not fit in memory will be written to disk.

The storage-level flags containing `DISK` (such as `MEMORY_AND_DISK`) enable this. By default, if partitions do not fit in memory, they will simply be evicted and will need to be recomputed when the persisted RDD is used (see [“LRU Caching”](#)). Therefore, persisting to disk can ensure that recomputation of those additional large partitions is avoided. However, reading from disk can be time-

intensive, so persistence to disk is only important if the cost of recomputation is particularly high.

TIP

It may be beneficial to allow writing to disk if you expect that an RDD cannot fit in memory. However, if the cost of recomputing the partitions is not high (they are simple mappings and don't reduce the size of the data) it may actually be faster to recompute some partitions rather than read from disk.

useMemory

If set, the RDD will be stored in-memory or be directly written to disk.

The `DISK_ONLY` storage levels are the only options that mark this as false. Most of the speed benefits of caching come from keeping RDDs in memory, so if the motivation for reuse is fast access for repeated computations, it is probably a good idea to choose a storage option that stores partitions in memory. However, there are some cases where disk-only persistence makes sense, e.g., when the computation is more expensive than reading in local disk or the network filesystem is especially slow (such as with certain object stores).

useOfHeap

If set, the RDD will be stored outside of the Spark executor in an external system such as Tachyon.

The storage option `off_heap` enables this property. If memory is a serious issue, or a cluster is noisy and partitions are evicted, this option may be compelling. We will talk more about the benefits of Tachyon in ["Alluxio \(nee Tachyon\)"](#).

deserialized

If set, the RDD will be stored as deserialized Java objects.

As we will discuss in ["Kryo"](#), this can make storing RDDs more space

efficient, especially when using a faster serializer—but incurs some performance overhead. Storage options that include the "`_SER`" suffix such as `MEMORY_ONLY_SER` enable serialization.

TIP

If your RDD is too large to persist in-memory, first try to serialize it with the `MEMORY_ONLY_SER` option. This will keep the RDD fast to access, but will decrease the memory needed to store it.

replication

Replication is an integer that controls the number of copies of the persisted data to be stored in the cluster.

By default this is set to 1; however, serialization options that end in `_2` such as `DISK_ONLY_2` replicate each partition across two nodes. Use this option to ensure faster fault tolerance. However, be aware that persistence with replication incurs double the space and speed costs of persistence without replication. Replication is usually only necessary in an instance of a noisy cluster or bad connection where failures are unusually likely. It might also be useful if you do not have time to recompute in case of failure, such as when serving a live web application.

NOTE

The RDD operation `cache()` is equivalent to the `persist()` operation with no storage level argument, i.e., `persist()`. Both `cache()` and `persist()` persist the RDD with the default storage-level `MEMORY_ONLY`, which is equivalent to `StorageLevel(false, true, false, true)`, which stores RDDs in-memory as deserialized Java objects, does not write to disk as partitions get evicted, and doesn't replicate partitions.

CHECKPOINTING

Checkpointing writes the RDD to an external storage system such as HDFS or S3, and—in contrast to persisting—forgets the RDD’s lineage. Since checkpointing requires writing the RDD outside of Spark, checkpointed information survives beyond the duration of a single Spark application and forces evaluation of an RDD. Checkpointing takes up more space in external storage and may be slower than persisting since it requires potentially costly write operations. However, it does not use any Spark memory and will not incur recomputation if a Spark worker fails.

Figure 5-2 illustrates the difference between in-memory persistence and checkpointing and RDD. Persisting stores the RDD’s partitions in-memory or on disk in the caching layer of each executor. Checkpointing writes each partition to some external system.

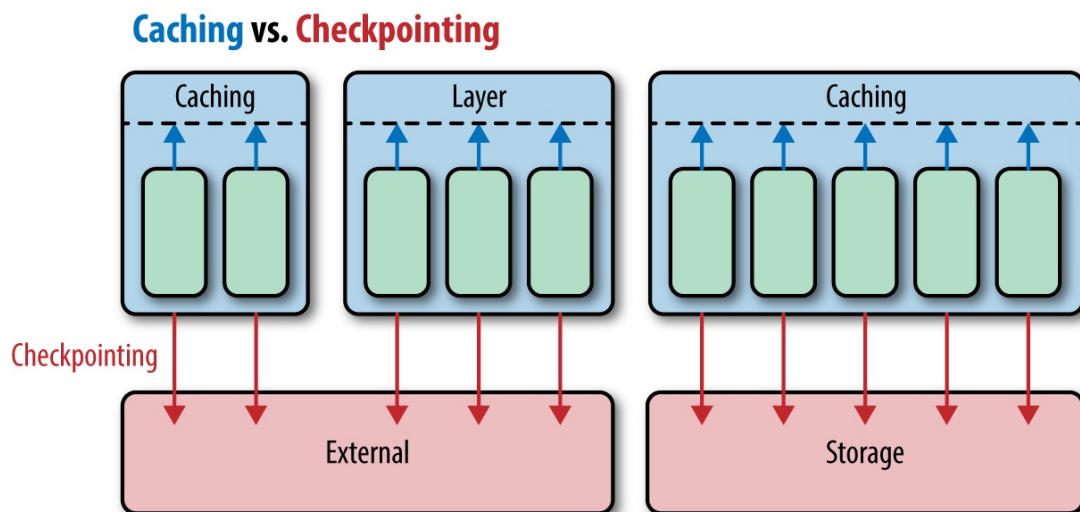


Figure 5-2. Caching versus checkpointing

It is best to use checkpointing when the cost of failure and recomputation is of more concern than additional space in external storage. Broadly speaking, we advise persisting when jobs are slow and checkpointing when they are failing. If a Spark job is failing due to out-of-memory

errors, checkpointing will reduce the cost and likelihood of failure without using up memory on the executors. If your jobs are failing due to network errors or preemption on a noisy cluster, checkpointing can reduce the likelihood of failure by breaking up a long-running job into smaller segments. To call `checkpoint`, call `setCheckpointDir(directory: String)` from the `SparkContext` object and pass in a path to a location on HDFS to write the intermediate results. Then, in the Spark job, call `.checkpoint()` from the RDD.

CHECKPOINTING EXAMPLE

Example 5-24 makes use of custom storage level and checkpointing options. The function is used in the Goldilocks example, which we describe in detail in “Goldilocks Version 4: Reduce to Distinct on Each Partition”, that makes use of custom storage level and checkpointing options. In this case we are doing several very expensive transformations: first a sort and then two very substantial map partitions routines. When running on a noisy cluster, we found it advantageous to checkpoint this function after the sort. The value of the `directory` parameter is the checkpoint directory. The `sorted` value is a sorted RDD or key/value pairs.

Example 5-24. Checkpoint example

```
def findQuantilesWithCustomStorage(valPairs: RDD[((Double, Int),
Long)],
    colIndexList: List[Int],
    targetRanks: List[Long],
    storageLevel: StorageLevel = StorageLevel.MEMORY_AND_DISK,
    checkPoint : Boolean, directory : String = ""): Map[Int,
Iterable[Double]] = {

    val n = colIndexList.last + 1
    val sorted = valPairs.sortByKey()
    if (storageLevel != StorageLevel.NONE) {
```

```

        sorted.persist(storageLevel)
    }

    if (checkPoint) {
        sorted.sparkContext.setCheckpointDir(directory)
        sorted.checkpoint()
    }

    val partitionColumnsFreq = getColumnsFreqPerPartition(sorted,
n)
    val ranksLocations = getRanksLocationsWithinEachPart(
        targetRanks, partitionColumnsFreq, n)
    val targetRanksValues = findTargetRanksIteratively(sorted,
ranksLocations)
    targetRanksValues.groupByKey().collectAsMap()
}

```

WARNING

Spark includes a *Local Checkpointing* option that truncates the RDD's lineage graph but doesn't persist to stable storage. This is not suitable for clusters that may experience failures, preemption, or dynamic scale-downs during the time the RDD may be referenced.

Alluxio (nee Tachyon)

Tachyon is a distributed, in-memory storage system that is developed separately from Spark. It sits above a storage system, such as S3 or HDFS, and can be used on its own or with an external computational framework such as Spark or MapReduce. Like Spark, Tachyon can be used in a standalone cluster mode, or with Mesos or YARN. Read more about Tachyon's architecture and how to integrate it with Spark in [the Tachyon documentation](#).

Tachyon can be used as an input or output source for Spark applications (data stored with Tachyon can be used to create RDDs) or for `off_heap`

persistence during a Spark application. Using Tachyon for persistence has several advantages. First, it reduces garbage collection overhead, since data is not stored as Java objects. Second, it allows multiple executors to share the same external memory pool in Tachyon. Third, since the data is stored in memory outside of Spark, it is not lost if individual executors crash. It can be particularly useful if you want to reuse an RDD but are running out of memory or seeing garbage collection errors. It is also the best way to reuse a very large RDD between multiple applications.

NOTE

Tachyon's developer and user communities are very strong in China, so part of its documentation may be stronger in Mandarin than in English.

LRU Caching

RDDs that are stored in memory and/or on disk in Spark are not automatically un-persisted when they are no longer going to be used downstream. Instead, RDDs stay in memory for the duration of a Spark application, until the driver program calls the function `unpersist`, or memory/storage pressure causes their eviction. Spark uses *Least Recently Used* or LRU caching, to determine which partitions to evict if the executors begin to run out of memory.

LRU caching dictates that the data structure that was least recently accessed will be evicted. However, because of lazy evaluation it may be a bit tricky to predict which partitions will be evicted first. Generally, Spark evicts the oldest partitions; those that were created or used in the earliest Spark job or in the earliest stage within a given job. (See [“Dividing the Space Within One Executor”](#) for a more detailed explanation of memory

management and partitions.) LRU caching behaves differently for different persistence options. For memory-only persistence operations configured with LRU caching, Spark will recompute the evicted partition each time it is needed. For memory and disk options, LRU caching will write the evicted partition to disk. If you want to take a persisted RDD out of memory to free up space, use `unpersist`.

SHUFFLE FILES

Regardless of a `persist` or `checkpoint` call, Spark does write some data to disk during a shuffle. These files are called “shuffle files” and they usually contain all of the records in each input partition sorted by mapper. Usually shuffle files remain in the local directory on the workers for the duration of an application. Thus if the driver program reuses an RDD that has already been shuffled, Spark may be able to avoid recomputing that RDD up to the point of shuffle by using the shuffle files on the mapper.

Unlike the other caches, we can’t determine if a given RDD still has its shuffle files present; e.g., there is no equivalent of the `isCheckpointed` command, which returns true if that RDD has been checkpointed. In general, though, shuffle files aren’t explicitly cleaned up until an RDD goes out of scope. However the web UI can be helpful in determining if stages are being skipped this way, as shown in [Figure 5-3](#).



Figure 5-3. Skipped stage from reading shuffle files

The performance of reusing shuffle files is similar to the performance of an RDD that is cached at the level of disk only.

WARNING

Shuffle files can be large, and Spark has no explicit cache management for them. Keeping references to RDDs depending on shuffled output can lead to out-of-disk errors if the RDDs are not garbage collected on the driver.

Out-of-disk-space errors can be unexpected, but in clusters with small amounts of disk space they are surprisingly common. Disk space errors can be caused by long-running shell environments in which RDDs created at the top scope are never garbage collected. Spark writes the output of its shuffle operations to files on the disk of the workers in the Spark local dir. These files are only cleaned up when an RDD is garbage collected, which if the amount of memory assigned to the driver program is large, can occur infrequently. One solution is to explicitly trigger garbage collection

(assuming the RDDs have gone out of scope)—if the DAG is getting too long, checkpointing can help make the RDDs available for garbage collection.

Noisy Cluster Considerations

Noisy clusters, or those with a high volume of unpredictable traffic, pose a fundamental challenge to Spark's evaluation. By default, Spark doesn't save most intermediate results (besides in a shuffle step). Thus, in the case of preemptions, Spark will have to recompute the calculation in the job up to the point of failure. In a noisy cluster, where long-running jobs are often interrupted, this poses a huge challenge. Checkpointing can be especially helpful to get jobs to run at all. Checkpointing breaks an RDD's lineage, therefore reducing the cost to recompute downstream transformations. Checkpointing also persists to external storage, so that unexpected failures do not lead to data loss. If failures are common but not fatal, it may be worth configuring your job to persist to multiple machines using a storage option like `MEMORY_AND_DISK_2`, which replicates data on two machines. That way, failures on one node will not require a recompute. This can be especially important with wide transformations, which are very expensive.

By default, Spark uses a first in, first out (FIFO) paradigm to queue jobs within a system. This means that the first job submitted will run in its entirety, getting priority on all the available resources. However, if a job doesn't need the whole cluster, the next job may start. FIFO scheduling can be useful to ensure that space-intensive jobs are able to use the resources that they need. However, if you launch a job a few seconds behind a many-hour process, the FIFO strategy can be frustrating. Spark offers a fair scheduler, modeled after the Hadoop fair scheduler, to allow

high-traffic clusters to share resources more evenly. The fair scheduler allocates the tasks from different jobs to the executors in a “round-robin fashion” (i.e., parsing out a few tasks to the executors from each job). With the fair scheduler, a short, small job can be launched before an earlier long-running job is completed.

The fair scheduler also supports putting jobs into pools and allocating different priority (weight) to those pools. Jobs within a pool are allocated the same number of resources, and the pools are allocated resources according to their weight. Using pools can be a good way to ensure that high-priority jobs or very expensive jobs are completed. The fair scheduler also ensures that users are allocated resources evenly regardless of how many jobs they submit. You can read more about using and configuring a fair scheduler in the [the Spark job scheduling documentation](#).

Interaction with Accumulators

The interaction of caching and accumulators can make reasoning about accumulators more difficult. As we mentioned, if part of an RDD has to be recomputed, Spark may continue to add values to the accumulator as it recomputes; causing the values in the recomputed part to be double counted. Furthermore, not all computations will always compute the entirety of a partition. Surprisingly, caching does not prevent either double counting or problems that arise from partially evaluated partitions. Cached partitions may be evicted, so double counting may still arise if the machine with the cached data fails or if the partition is evicted to make space for a more recently cached partition. Unfortunately, caching with accumulators may cause a job that appears to compute the correct value on small data to later compute the incorrect value on large data.

Conclusion

Now that you have explored how to get the most out of your standard RDD transformations, as well as joins, it's time to explore the concerns associated with the most important and complicated subset of RDD transformations, key/value pair operations. Not all of the techniques you will have learned need to be applied in every Spark program, and some of the takeaways from this chapter are more about when certain tools are not a good fit (see “[Accumulators](#)”). Many of the same techniques and considerations for standard RDD transformations apply when working with key/value data: if your transformation doesn't depend on the key, the techniques from this chapter may even be more relevant.

¹ If A and B are sets, $(A - B) \cup (B \cap A) = A$ in all cases. This is not true in Spark. If `rddA` or `rddB` have duplicate keys or if `rddA` and `rddB` have overlapping keys, then $(A - B) \cup (B \cap A)$ is a *subset* of A.

² Originally planned for 2.0.

³ Some notable exceptions are inside of certain ML algorithms, which if passed in an unpersisted RDD will automatically persist and unpersist the RDD.